

Lecture 7

Single Inheritance

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem

Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Single Inheritance

- **Single inheritance:** OO mechanism that allows *subclassing* of single class
- **Subclassing:** Allowing of a class to inherit properties and methods from another
 - *Derived class* inherits members of *base class*
 - Allows *reusing behavior* of base class by derived class
- Subclassing implies subtyping in Python (`Child <: Parent`)
 - Subclass / derived / child class $\Rightarrow$ Subtype
 - Superclass / base / parent class $\Rightarrow$ Supertype

Single Inheritance

```
class Base:
    def hello(self):
        print('Hello, world!')

class Derived(Base): # `Derived` inherits `hello`
    def hi(self):     # of `Base`; `Derived <: Base`
        print('Hi!') # means `Derived` "is a" `Base`

d = Derived()
d.hi()          # Hi!
d.hello()       # Hello, world!
```

Single Inheritance

```
class Dog:
    def bark(self):
        print('Arf arf!')

# `TrainedDog <: Dog`; `TrainedDog` "is a" `Dog`
class TrainedDog(Dog):
    def sit(self):
        print('*sits down*')

d = TrainedDog()
d.bark()           # Arf arf!
```

Method Overriding

- Subclasses can *override* methods inherited from superclasses:

07/overriding

Subset of parent class behavior can be modified via overriding

Method Resolution Order

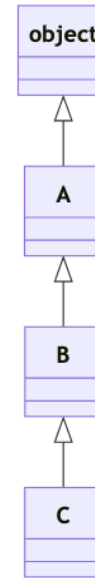
- For single inheritance: inheritance chain is traversed in order (direct superclass) until method is found

```
class A:
    def f(self, x: int, y: int):
        print(x + y)

class B(A):
    pass

class C(B):
    pass

c = C()
c.f(11, 1)  # 12
```



Method Resolution Order

- `object` is an implicit superclass of all Python objects
- For *multiple inheritance* (will not be covered in CS 12):
<https://docs.python.org/3/howto/mro.html>

Initializers are Methods

- `__init__` of parent class is inherited just like any other method:

```
class A:  
    def __init__(self, n: int):  
        self.n: int = n
```

```
class B(A):  
    pass
```

```
b = B(12)  # Has same constructor params as A  
print(b.n)  # Exists due to inherited `__init__`
```

Self-check

Determine the output of the following:

```
class A:
    def __init__(self, a: int):
        self.attr_of_a = a

class B(A):
    def __init__(self, b: int):
        self.attr_of_b = b

b = B(12)
print(b.attr_of_b); print(b.attr_of_a)
```

Initializer Overloading Caveat

Child class can override inherited `__init__` just like any other method:

```
07/child_overriding
```

Overriding `__init__` means no other `__init__` in inheritance chain is called; *fields they initialize will be absent*

How do you ensure that all `__init__`s in inheritance chain are called?

Initializer Overloading Caveat

`super()` allows access to members of parent class in child class

- Use `super()` to call `__init__` of superclass:

07/super

Interface Inheritance

- **Interface inheritance:** Only interface (method signatures) is inherited (behaviors excluded; no code sharing)
 - Contrast with *implementation inheritance* (behaviors are also inherited)
- Implicit interface subtyping via Protocol:

07/implicit_interface

Interface Inheritance

- **Interface inheritance:** Only interface (method signatures) is inherited (behaviors excluded; no code sharing)
 - Contrast with *implementation inheritance* (behaviors are also inherited)
- Explicit interface subtyping via *interface inheritance*:

07/explicit_interface

Abstract Base Class

- **Abstract base class:** Class with both *already implemented methods* and *methods to be implemented*
 - Meant to be inherited from by subclasses
 - Cannot be instantiated on its own (just like interfaces)

07/abc

Composition Over Inheritance

Misguided tendency of students: Designing everything using implementation inheritance as default tool (modeling everything using “is-a”)

CS 12 deliberately covers *composition before inheritance* to stress this point

Composition Over Inheritance

- Use implementation inheritance only if:
 - There is a need for code sharing among variants
 - Resulting class hierarchy is shallow and narrow
 - Subclasses have little to no dependencies
 - Behavioral subtyping rules (future lec) can be satisfied

Composition Over Inheritance

- Extra care must be taken when using implementation inheritance
 - Can easily end up with hard-to-maintain or brittle code (future lec)
- No problem with interface inheritance
 - Newer PLs like Rust and Go do not have implementation inheritance
 - Forced to use composition (“has-a”)

Self-check

Determine the output of the code below without running it:

```
07/inheritance_self_check
```

Self-check

Inheritance allows selective code sharing and overriding of Grader by subclasses:

07/grader

Grader also does not need to be modified to introduce variations in behavior:

```
average_grader = Grader(...)  
cancel_the_lowest_grader = CancelTheLowestGrader(...)  
floor_grader = FloorGrader(...)
```

Self-check

Create a class `ExtremistGrader` that is a subclass of `Grader`, but:

- Returns `"1.00"` for grades of 99 or better
- Returns `"3.00"` for grades in the range `[60, 99)`
- Returns `"5.00"` otherwise
- Additionally, finish the implementation of `transform_raw` in `Grader`
 - You may refer to the CS 12 grading scale for this (no `"4.00"`, no `"INC"`)

Self-check

Create a class `StepUpGrader` that is a subclass of `Grader`, but:

- Takes an extra int constructor parameter n :
 - n means all passing grades are raised n steps (to at most "1.00")
 - $n = 1$: Raised one step (e.g., "1.25" to "1.00")
 - $n = 2$: Raised two steps (e.g., "2.75" to "2.25")
 - $n < 1$: No effect
- n should not raise grades that are "5.00"

Back to Motivating Example

Composition-based solution for Part 2 of <https://ncase.me/trust/>:

07/trust_composition

Composition: Player “has a” CopycatStrategy

Back to Motivating Example

Inheritance-based solution for Part 2 of <https://ncase.me/trust/>:

07/trust_inheritance

Composition: CopycatPlayer “is a” Player